gotchas. First, middleware can modify a response generated farther down the pipeline only if the response *hasn't yet been sent to the client*. This situation can be a problem if, for example, an error occurs while ASP.NET Core is sending a static file to a client. In that case, ASP.NET Core may start streaming bytes to the client immediately for performance reasons. When that happens, the error-handling middleware won't be able to run, as it can't reset the response. Generally speaking, you can't do much about this problem, but it's something to be aware of.

A more common problem occurs when the error-handling path throws an error during the reexecution of the pipeline. Imagine that there's a bug in the code that generates the menu at the top of the page in a Razor Pages app:

1. When the user reaches your home page, the code for generating the menu bar throws an exception.
2. The exception propagates up the middleware pipeline.
3. When reached, `ExceptionHandlerMiddleware` captures it, and the pipe is reexecuted, using the error-handling path.
4. When the error page executes, it attempts to generate the menu bar for your app, which again throws an exception.
5. The exception propagates up the middleware pipeline.
6. `ExceptionHandlerMiddleware` has already tried to intercept a request, so it lets the error propagate all the way to the top of the middleware pipeline.
7. The web server returns a raw 500 error, as though there were no error-handling middleware at all.